



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
APLICACIONES WEB

“Práctica en clase Instalación de los entornos y primeros script PHP y PERL”

ESTUDIANTE:

FRED ADRIÁN BELTRÁN MONTIEL

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

QUINTO SEMESTRE “A”

PPA 2026-2027

1. Instalación y verificación del entorno

A continuación, se presentan las capturas de pantalla que evidencian la correcta instalación y configuración del entorno de desarrollo XAMPP con PHP 8.2 y Perl 5.

1.1 Panel de control XAMPP con Apache activo

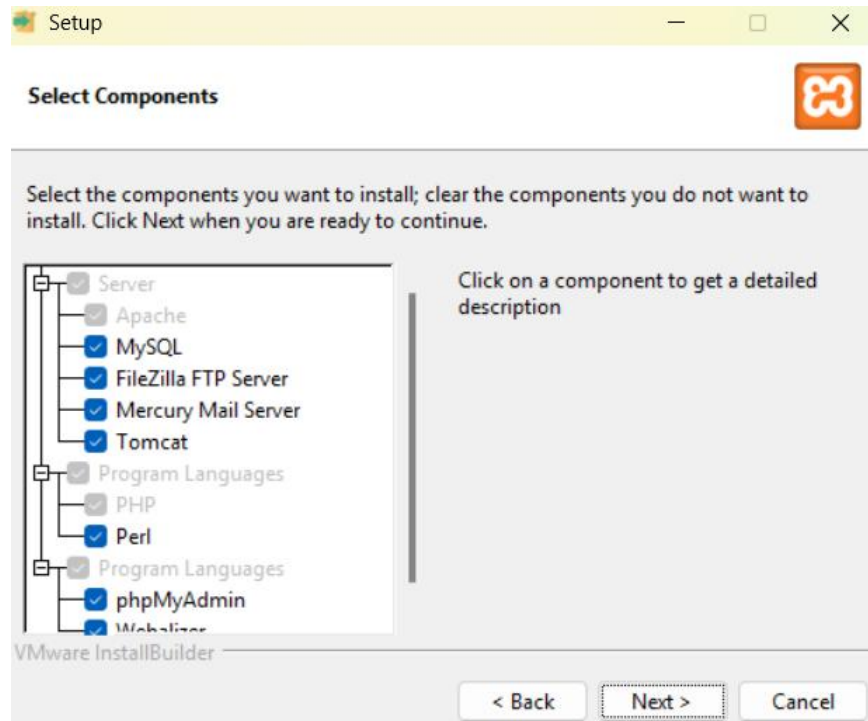


Figura 1. Setup de XAMPP (Se incluyen los componentes Apache, PHP y PERL)

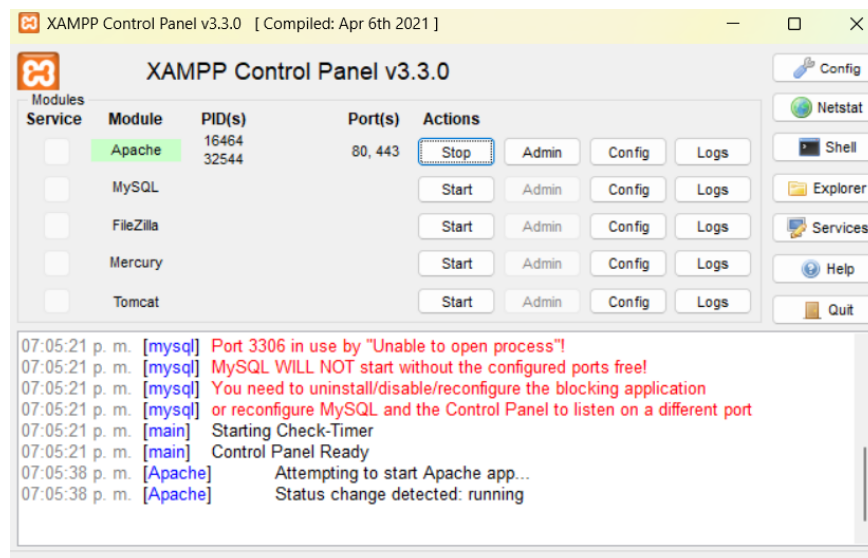
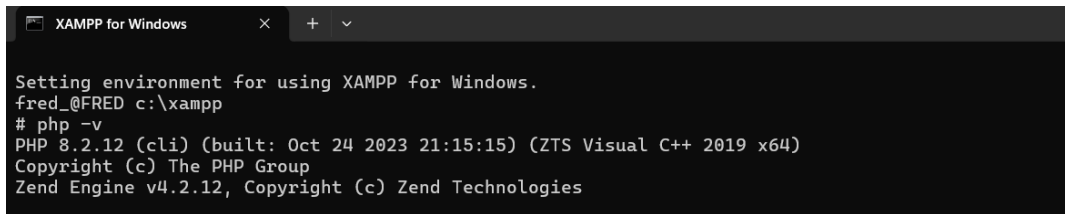


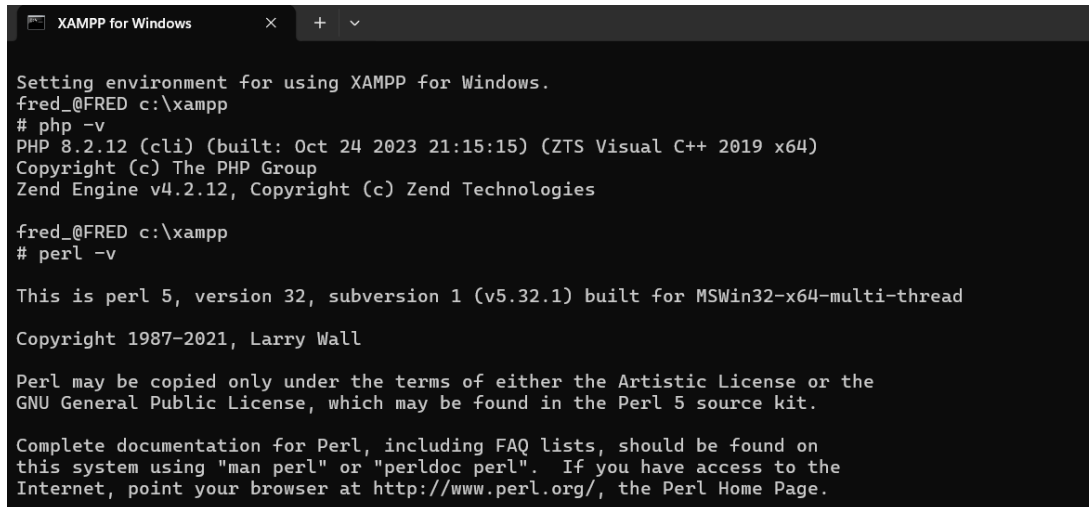
Figura 2. Panel de Control XAMPP con Apache running

1.2 Verificación de versiones en terminal



```
XAMPP for Windows
Setting environment for using XAMPP for Windows.
fred_@FRED c:\xampp
# php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
```

Figura 3. Se verifica la version PHP 8.2.12 en el cmd



```
XAMPP for Windows
Setting environment for using XAMPP for Windows.
fred_@FRED c:\xampp
# php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

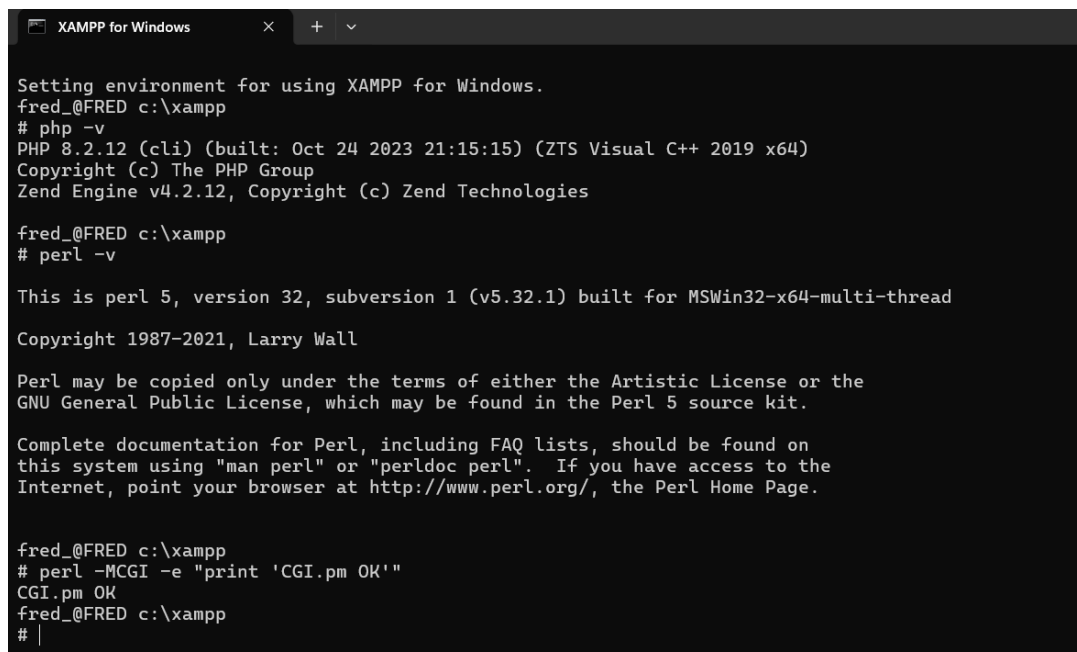
fred_@FRED c:\xampp
# perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.
```

Figura 4. Verificación de Perl 5.32.1 en el cmd



```
XAMPP for Windows
Setting environment for using XAMPP for Windows.
fred_@FRED c:\xampp
# php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

fred_@FRED c:\xampp
# perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.

fred_@FRED c:\xampp
# perl -MCGI -e "print 'CGI.pm OK'"
CGI.pm OK
fred_@FRED c:\xampp
#
```

Figura 5. Se verifica el CGI.pm en el cmd

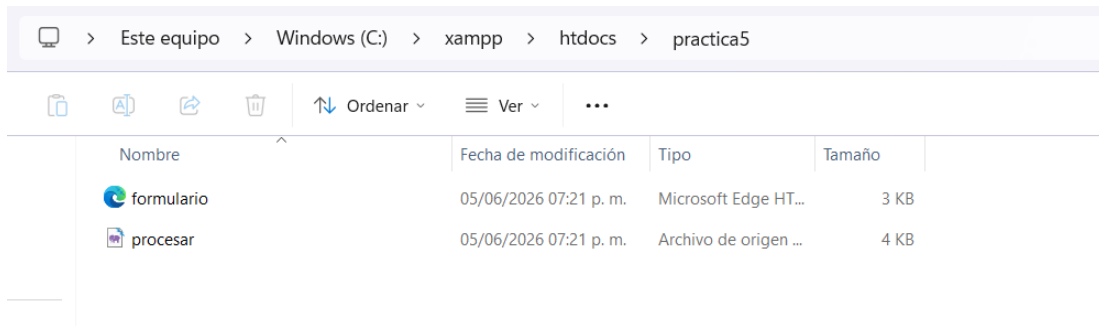


Figura 6. Evidencia de la creación de los archivos formulario.html y procesar.php con sus respectivos codes

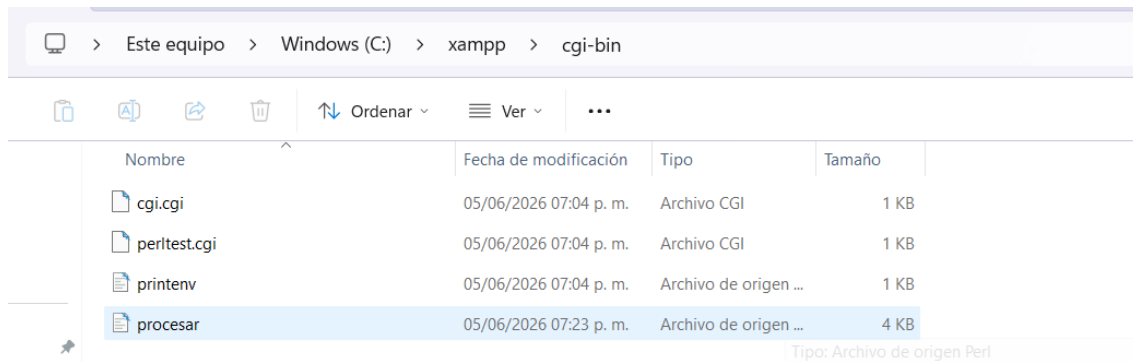


Figura 7. Evidencia de la creación del archivo procesar.perl en el cgi-bin

2. Formulario HTML en el navegador

El formulario formulario.html fue creado en la carpeta htdocs/practica5/ y se visualiza correctamente en el navegador accediendo a <http://localhost/practica5/formulario.html>.



Figura 8. Captura del http://localhost funcionando correctamente

localhost/practica5/formulario.html

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *
Ej: Maria Garcia

Correo electronico *
usuario@dominio.com

Edad (1-120) *
Ej: 22

Mensaje *
Escribe tu mensaje aqui...

Enviar con PHP Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 9. Captura del formulario.html con los botones respectivos y funcionando correctamente

3. Evidencia de los 10 casos de prueba

Cada caso fue probado enviando el formulario tanto con el backend PHP como con PERL/CGI. A continuación, se presenta la evidencia de cada caso.

Caso 1 — Campos vacíos

Datos ingresados: formulario enviado sin rellenar ningún campo.

Resultado esperado: error en los 4 campos (nombre, email, edad, mensaje).

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Nombre: El nombre no puede estar vacío.

Email: El correo electrónico no tiene un formato válido.

Edad: La edad debe ser un número entero entre 1 y 120.

Mensaje: El mensaje no puede estar vacío.

[Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: -1

Figura 10. Captura de resultado PHP con errores en los 4 campos

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un numero entero entre 1 y 120.

Email: El correo electronico no tiene formato valido.

Mensaje: El mensaje no puede estar vacio.

Nombre: El nombre no puede estar vacio.

[Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 11. Captura de resultado PERL con errores en los 4 campos

Caso 2 — Email sin dominio

Datos: nombre=Ana, email=ana@, edad=20, mensaje=Holaaa

Resultado esperado: error solo en el campo email.

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana

Correo electronico *

ana@

Edad (1-

20



Escribe una parte después del símbolo "@". "ana@" está incompleto.

Mensaje *

Holaaa

Enviar con PHP

Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 12. Captura de resultado (no permite envío del formulario)

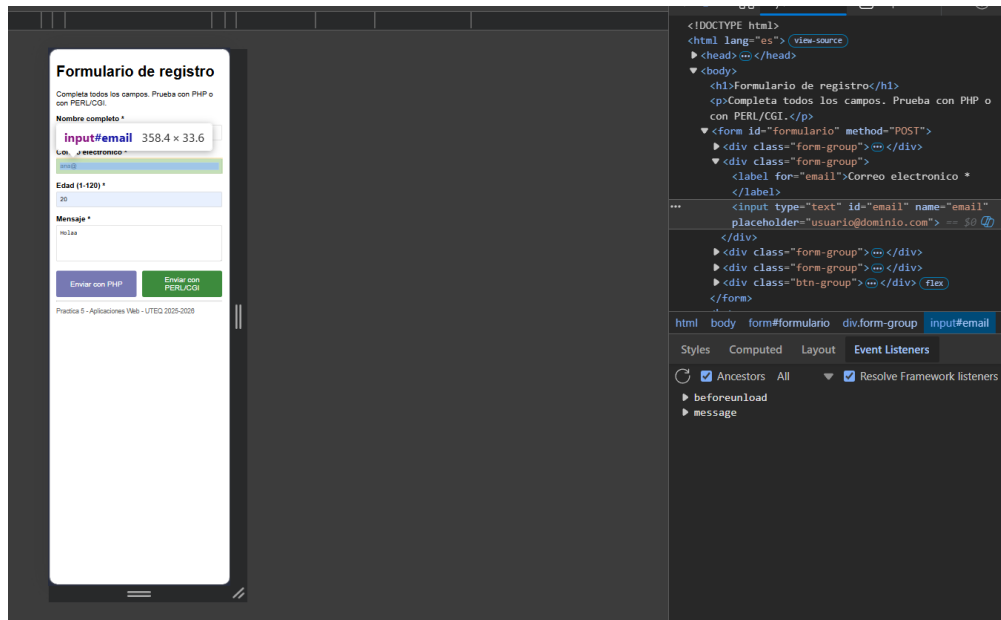


Figura 13. Captura de modificación del input type email para verificar el php y perl

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Email: El correo electronico no tiene un formato valido.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 14. Captura de resultado de error en email por formato no válido en PHP

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Email: El correo electronico no tiene formato valido.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 15. Captura de resultado de error en email por formato no válido en PERL

Caso 3 — Email sin arroba

Datos: email=anahotmail.com, demás campos válidos.

Resultado esperado: error solo en el campo email.

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana

Correo electronico *

anahotmail.com

Incluye "@" en la dirección de correo electrónico. En "anahotmail.com" falta un símbolo "@".

Mensaje *

Holaaa

Enviar con PHP Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 16. Verificación de que el input detecta falta de arroba, posteriormente se modificará para perl y php

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Email: El correo electronico no tiene un formato valido.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 17. Captura de resultado de error en email por formato no válido en PHP

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Email: El correo electronico no tiene formato valido.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 18. Captura de resultado de error en email por formato no válido en PERL

Caso 4 — Edad negativa

Datos: edad=-20, demás campos válidos.

Resultado esperado: error solo en el campo edad (fuera del rango 1–120).

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana

Correo electrónico *

ana@hotmail.com

Edad (1-120) *

-20

Mensaje *

Hola

! El valor debe ser mayor o igual que 1.

Enviar con PHP

Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 19. Verificación de que el input detecta edad negativa, posteriormente se modificará para perl y php

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un numero entero entre 1 y 120.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 20. Captura de resultado PHP donde detecta que la edad no está dentro del rango

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un numero entero entre 1 y 120.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 21. Captura de resultado PERL donde detecta que la edad no está dentro del rango

Caso 5 — Edad demasiado alta

Datos: edad=200, demás campos válidos.

Resultado esperado: error solo en el campo edad (fuera del rango 1–120).

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana

Correo electrónico *

ana@hotmail.com

Edad (1-120) *

200

Mensaje *

Hola

El valor debe ser menor o igual que 120.

Enviar con PHP

Enviar con PERL/CGI

Práctica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 22. Verificación de que el input detecta edad fuera de rango, posteriormente se modificará para perl y php

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

Figura 23. Captura de resultado PHP donde detecta que la edad no está dentro del rango

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

Figura 24. Captura de resultado PERL donde detecta que la edad no está dentro del rango

Caso 6 — Edad no numérica

Datos: edad=e, demás campos válidos.

Resultado esperado: error solo en edad (debe ser entero).

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana

Correo electrónico *

ana@hotmail.com

Edad (1-120) *

e

Mensaje *

Hola!!!

! Escribe un número.

Enviar con PHP

Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 25. Verificación de que el input detecta formato no válido, posteriormente se modificará para perl y php

PHP 8.2

Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un numero entero entre 1 y 120.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 26. Captura de resultado PHP donde detecta que la edad no está en el formato

PERL/CGI

Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un numero entero entre 1 y 120.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 27. Captura de resultado PERL donde detecta que la edad no está en el formato

Caso 7 — Intento XSS en nombre

Datos: nombre=<script>alert(1)</script>, demás campos válidos.

Resultado esperado: el texto se muestra escapado como <script>... sin ejecutarse.

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	<script>alert(1)</script>
Email	ana@hotmail.com
Edad	20 anos
Mensaje	Holaa

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 28. Resultado PHP donde ignora el script como debe

```
Ajuste de línea
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8">
5 <title>Resultado PHP</title>
6 <style>
7 body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
8 .error { color: #c0392b; background: #fdecea; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
9 .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1e6b3c; margin: 4px 0; }
10 table { width: 100%; border-collapse: collapse; }
11 td, th { padding: 8px 12px; border: 1px solid #ddd; text-align: left; }
12 th { background: #7779b3; color: white; }
13 tr:nth-child(even) { background: #f5f5f5; }
14 .badge { display: inline-block; padding: 2px 10px; border-radius: 12px; font-size: 0.8rem; background: #7779b3; color: white; }
15 </style>
16 </head>
17 <body>
18 <h1><span class="badge">PHP 8.2</span> Resultado del procesamiento</h1>
19
20 <div class="exito"><strong>Datos recibidos y validados correctamente.</strong></div>
21 <h2>Resumen de los datos:</h2>
22 <table>
23 <tr><th>Campo</th><th>Valor recibido</th></tr>
24 <tr><td>Nombre</td><td>&lt;script&gt;alert(1)&lt;/script&gt;</td></tr>
25 <tr><td>Email</td><td>ana@hotmail.com</td></tr>
26 <tr><td>Edad</td><td>20 anos</td></tr>
27 <tr><td>Mensaje</td><td>Holaa</td></tr>
28 </table>
29
30 <p><a href="formulario.html">&larr; Volver al formulario</a></p>
31 <hr>
32 <p style="font-size:0.8rem; color:#888;">
33   Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1 </p>
34 </body>
35 </html>
```

Figura 29. Código fuente PHP — confirma entidades HTML (<script>)

PERL/CGI

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	<script>alert(1)</script>
Email	ana@hotmail.com
Edad	20 anos
Mensaje	Holaa

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 30. Resultado PERL donde ignora el script como debe

```
Ajuste de línea ☐
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Resultado PERL/CGI</title>
6   <style>
7     body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
8     .error { color: #c0392b; background: #fdecea; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
9     .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1e6b3c; }
10    table { width:100%; border-collapse: collapse; }
11    td, th { padding:8px 12px; border:1px solid #ddd; }
12    th { background:#3d8b3d; color:white; }
13    tr:nth-child(even) { background:#f5f5f5; }
14    .badge { display:inline-block; padding:2px 10px; border-radius:12px; font-size:0.8rem; background:#3d8b3d; color:white; }
15  </style>
16 </head>
17 <body>
18 <h1><span class="badge">PERL/CGI</span> Resultado del procesamiento</h1>
19 <div class="exito"><strong>Datos recibidos y validados correctamente.</strong></div>
20 <h2>Resumen de los datos:</h2>
21 <table>
22 <tr><th>Campo</th><th>Valor recibido</th></tr>
23 <tr><td>Nombre</td><td>&lt;script&gt;alert(1)&lt;/script&gt;</td></tr>
24 <tr><td>Email</td><td>ana@hotmail.com</td></tr>
25 <tr><td>Edad</td><td>20 anos</td></tr>
26 <tr><td>Mensaje</td><td>holaa</td></tr>
27 </table>
28 <p><a href="/practica5/formulario.html">&larrr; Volver</a></p>
29 <hr>
30 <p style="font-size:0.8rem; color:#888;">Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST</p>
31 </body>
32 </html>
33
```

Figura 31. CAPTURA: Código fuente PERL — confirma entidades HTML

Caso 8 — Intento XSS en mensaje

Datos: mensaje=, demás campos válidos.

Resultado esperado: la etiqueta se muestra como texto plano sin ejecutar el evento.

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	ana@hotmail.com
Edad	20 anos
Mensaje	

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 32. Resultado PHP donde ignora el script como debe

```
Ajuste de línea
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Resultado PHP</title>
6   <style>
7     body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
8     .error { color: #c0392b; background: #fdecea; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
9     .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1e6b3c; margin: 4px 0; }
10    table { width: 100%; border-collapse: collapse; }
11    td, th { padding: 8px 12px; border: 1px solid #ddd; text-align: left; }
12    th { background: #7779b3; color: white; }
13    tr:nth-child(even) { background: #f5f5f5; }
14    .badge { display: inline-block; padding: 2px 10px; border-radius: 12px; font-size: 0.8rem; background: #7779b3; color: white; }
15  </style>
16 </head>
17 <body>
18   <h1><span class="badge">PHP 8.2</span> Resultado del procesamiento</h1>
19
20   <div class="exito"><strong>Datos recibidos y validados correctamente.</strong></div>
21   <h2>Resumen de los datos:</h2>
22   <table>
23     <tr><th>Campo</th><th>Valor recibido</th></tr>
24     <tr><td>Nombre</td><td>Ana</td></tr>
25     <tr><td>Email</td><td>ana@hotmail.com</td></tr>
26     <tr><td>Edad</td><td>20 anos</td></tr>
27     <tr><td>Mensaje</td><td>&lt;img src=x onerror=alert(1)&gt;</td></tr>
28   </table>
29
30   <p><a href="formulario.html">&laquo; Volver al formulario</a></p>
31   <hr>
32   <p style="font-size:0.8rem; color:#888;">
33     Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1 </p>
34 </body>
35 </html>
```

Figura 33. Código fuente PHP — confirma entidades HTML

PERL/CGI

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	ana@hotmail.com
Edad	20 anos
Mensaje	

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 34. Resultado PERL donde ignora el script como debe

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Resultado PERL/CGI</title>
6   <style>
7     body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
8     .error { color: #c0392b; background: #fdecea; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
9     .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1e6b3c; }
10    table { width:100%; border-collapse: collapse; }
11    td, th { padding:8px 12px; border:1px solid #ddd; }
12    th { background:#3d8b3d; color:white; }
13    tr:nth-child(even) { background:#f5f5f5; }
14    .badge { display:inline-block; padding:2px 10px; border-radius:12px; font-size:0.8rem; background:#3d8b3d; color:white; }
15  </style>
16 </head>
17 <body>
18   <h1><span class="badge">PERL/CGI</span> Resultado del procesamiento</h1>
19   <div class="exito"><strong>Datos recibidos y validados correctamente.</strong></div>
20   <h2>Resumen de los datos:</h2>
21   <table>
22     <tr><th>Campo</th><th>Valor recibido</th></tr>
23     <tr><td>Nombre</td><td>Ana</td></tr>
24     <tr><td>Email</td><td>ana@hotmail.com</td></tr>
25     <tr><td>Edad</td><td>20 anos</td></tr>
26     <tr><td>Mensaje</td><td>&lt;img src=x onerror=alert(1)&gt;</td></tr>
27   </table>
28   <p><a href="/practica5/formulario.html">&larr; Volver</a></p>
29   <hr>
30   <p style="font-size:0.8rem; color:#888;">Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST</p>
31 </body>
32 </html>

```

Figura 35. CAPTURA: Código fuente PERL — confirma entidades HTML

Caso 9 — Datos completamente válidos

Datos: nombre=Maria , email=maria@gmail.com, edad=22, mensaje=Este es mensaje de prueba.

Resultado esperado: resumen de los datos válidos sin errores.

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria
Email	maria@hotmail.com
Edad	22 anos
Mensaje	Este es mensaje de prueba

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 36. Captura de resultado PHP

PERL/CGI

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria
Email	maria@hotmail.com
Edad	22 anos
Mensaje	Este es mensaje de prueba

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 37. Captura de resultado PERL

Caso 10 — Nombre con caracteres especiales válidos

Datos: nombre=Maria Jose Munoz-Lopez, demás campos válidos.

Resultado esperado: datos aceptados y mostrados correctamente.

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria Jose Munoz-Lopez
Email	maria@hotmail.com
Edad	22 anos
Mensaje	Holaa

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

Figura 38. Resultado PHP

PERL/CGI

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria Jose Munoz-Lopez
Email	maria@hotmail.com
Edad	22 anos
Mensaje	Holaa

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

Figura 39. Resultado PERL

4. Tabla comparativa PHP 8.2 vs. PERL 5.38 con CGI.pm

La siguiente tabla resume las diferencias observadas durante la práctica entre PHP 8.2 y PERL 5.38 con CGI.pm. Se ha añadido una columna de observación personal basada en la experiencia directa con ambos scripts.

Criterio	PHP 8.2	PERL 5.38 con CGI.pm	Observación personal
Integración con Apache	Módulo nativo (mod_php): el intérprete vive dentro del proceso de Apache, sin crear procesos nuevos por petición.	CGI clásico: Apache lanza un proceso Perl nuevo por cada petición, lo que genera mayor overhead.	PHP se siente más fluido porque no hay ese costo de arrancar un proceso nuevo cada vez.
Leer datos del formulario	<code>filter_input(INPUT_POST, 'campo', FILTRO)</code> o <code>\$ POST['campo']</code>	<code>\$cgi->param('campo')</code> mediante objeto CGI->new	PHP es más directo. En Perl hay que instanciar el objeto CGI primero, lo cual es más verboso.
Función de saneamiento XSS	<code>htmlspecialchars(\$val, ENT_QUOTES, 'UTF-8')</code>	<code>CGI::escapeHTML(\$val)</code>	Ambas hacen lo mismo. La de Perl es más corta de escribir pero menos conocida.
Validación de email	<code>filter_input(..., FILTER_VALIDATE_EMAIL)</code> — filtro integrado en el núcleo de PHP	Expresión regular manual: <code>/^[...]+\@[...]+\.[a-zA-Z]{2,}\$/</code>	Me pareció una ventaja de PHP tener el filtro ya incorporado, en Perl hay que escribirlo a mano.
Validación de entero	<code>FILTER_VALIDATE_INT</code> con opciones de rango	Expresión regular <code>/\d+/</code> + comparación numérica	PHP nuevamente gana en comodidad. En Perl la validación requiere más líneas.
Cabecera HTTP	No es obligatorio emitirla manualmente; <code>header()</code> solo se usa para redirecciones.	Obligatorio como primera salida: <code>\$cgi->header()</code> antes de cualquier <code>print</code> .	Al principio esto me confundió en Perl. Si olvidas el <code>header()</code> da error 500 sin explicación clara.
Modo de tipos estrictos	<code>declare(strict_types=1)</code> en PHP 8	<code>use strict;</code> y <code>use warnings;</code>	Similar concepto, diferente sintaxis. Ambos ayudan a evitar errores difíciles de detectar.
Salida HTML	Intercalado PHP/HTML con bloques <code><?php></code> o con <code>print</code>	<code>print</code> , heredoc o <code>CGI->start_html()</code>	El heredoc de Perl es cómodo para bloques grandes de HTML, aunque la sintaxis al principio asusta.

Permisos de archivo	No requiere permisos especiales en Windows/XAMPP	chmod 755 obligatorio en Linux/Mac (cgi-bin)	En Windows con XAMPP no noté diferencia, pero en Linux sería un paso extra que podría causar confusión.
Curva de aprendizaje	Media: sintaxis intuitiva, documentación extensa en español	Alta: sintaxis expresiva pero densa; muchas formas de hacer lo mismo	PHP fue notablemente más fácil de entender desde cero. Perl requiere más tiempo de adaptación.
Casos de uso actuales	Desarrollo web dinámico, CMS (WordPress, Drupal), APIs REST	Administración de sistemas, bioinformática, automatización de texto	Para proyectos web modernos PHP es la elección obvia. Perl brilla más en scripting y automatización.

5. Reflexión final

Durante esta práctica tuve la oportunidad de trabajar con dos tecnologías de servidor con filosofías muy distintas: PHP 8.2 y PERL 5.38 con CGI.pm. La diferencia que me pareció más significativa fue la forma en que cada lenguaje integra la seguridad en el procesamiento de formularios. PHP tiene funciones como `filter_input()` y `FILTER_VALIDATE_EMAIL` integradas en su núcleo, lo que hace que validar datos sea más intuitivo y requiera menos código. En Perl, en cambio, la validación de email se realiza mediante expresiones regulares escritas manualmente, lo cual da más control, pero también más responsabilidad al programador y mayor posibilidad de errores si la expresión regular está mal construida.

Otra diferencia importante fue el comportamiento frente al servidor Apache. PHP funciona como módulo nativo, lo que significa que no hay overhead de creación de procesos por cada petición. Perl con CGI clásico lanza un proceso nuevo por cada solicitud, lo que en un entorno de alta concurrencia representaría un problema de rendimiento notable.

En cuanto a cuándo elegiría uno u otro para un proyecto real: optaría por PHP para cualquier aplicación web dinámica moderna, especialmente si el equipo no tiene experiencia previa en scripting avanzado, ya que su curva de aprendizaje es menor y su documentación en español es muy completa. Elegiría Perl en contextos de automatización de sistemas, procesamiento masivo de texto o proyectos en el ámbito de la bioinformática, donde históricamente ha demostrado ser muy eficiente y donde la expresividad de su sintaxis es una ventaja real.